STAGE 1

In this stage I build a tokenizer from scratch for data preparation and sampling purposes.

Here for dataset I am using WikiText-2 from huggingface and TinyStories.

Steps followed usually :

1. Split the text into individual word and subwords
2. Convert these tokens into token IDs.

```
! pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from data
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from dat
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from hugging
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from htt
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyth
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from ma
```

```
from datasets import load_dataset

wiki = load_dataset("wikitext", "wikitext-2-raw-v1")
print(wiki)
```

```
DatasetDict({
    test: Dataset({
        features: ['text'],
        num_rows: 4358
    })
    train: Dataset({
        features: ['text'],
        num_rows: 36718
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 3760
    })
})
```

We can see here its a dataset dict. For encoder we only need the train part.

```
wiki_text = wiki['train']['text']
print(repr(wiki_text[0]))
```

```
''
```

```
for i in range(5):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyr
" The game began development in 2010 , carrying over a large portion of the work done on Valk
```

Alright first i thought something was wrong but it seems everything is working correctly.

Looks good to me now i think i can move forward.

```
for i in range(10):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyr
" The game began development in 2010 , carrying over a large portion of the work done on Valk
" It met with positive sales in Japan , and was praised by both Japanese and western critics
''
' = = Gameplay = = \n'
''
" As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@
```

It seems as if theres a lot of empty spaces i should clean them first.

```
wiki_clean = [item for item in wiki_text if item.strip() != ' ']
for i in range(10):
  print(wiki_clean[i])
```

```
 = Valkyria Chronicles III =

 Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyri

 The game began development in 2010 , carrying over a large portion of the work done on Valky

 It met with positive sales in Japan , and was praised by both Japanese and western critics .

 = = Gameplay = =

 As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@
```

Ideas

I am keeping the *= = = Titles = = =* sections in hopes my model will learn these mean headings or titles and might print

= = = Knight's Voyage = = =

or something like that

```
"dataset = wiki_clean + stories_text # I forgot to convert both types hehe :p "
```

```
'dataset = wiki_clean + stories_text # I forgot to convert both types hehe :p '
```

```
print(len(dataset))
```

```
2150225
```

Sweet

Now we build the tokenizer.

But before that I will make a .py file.

PS: Now I think i should remove " *==== Titles ===* "

I can include them in finetuning

```
dataset = [item for item in dataset if not item.strip().startswith("=")]
```

```
#A pretty stupid line
print("Searching if dataset has lines starting with '= = = Title = = =' ")
```

```
for i in range(len(dataset)):
  if dataset[i].strip().startswith("="):
    print(dataset[i])

print(". . .\n")

print("Searching complete lets look at some samples")
for i in range(5):
  print(dataset[i])
```

```
Searching if dataset has lines starting with '= = = Title = = ='
. . .

Searching complete lets look at some samples


 Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyri

 The game began development in 2010 , carrying over a large portion of the work done on Valky

 It met with positive sales in Japan , and was praised by both Japanese and western critics .
```

LOL it didnt print anything but contents did print stuff

For the time being i save my dataset

```
"""final_dataset = " ".join(dataset)
print(final_dataset[1:9])"""
```

```
'final_dataset = " ".join(dataset)\nprint(final_dataset[1:9])'
```

```
"""print(final_dataset[:100])"""
```

```
'print(final_dataset[:100])'
```

tiktoken is a library that helps us use BPE. We instantiate BPE.

```
# ! pip install tiktoken
```

```
"""import importlib
import tiktoken"""
```

```
'import importlib\nimport tiktoken'
```

```
"""tokenizer = tiktoken.get_encoding("gpt2")"""
```

```
'tokenizer = tiktoken.get_encoding("gpt2")'
```

Every tokenizer has encode method and decode method.

This is good enough but I'd like to make my own tokenizer.

```
print(len(dataset))
```

```
2150225
```

```
#STEP 1
vocab = []

for line in range(394999):
    words = dataset[line].split()
    for word in words:
        vocab.append(" ".join(list(word)) + " </w>")
```

```
print(len(vocab))
```

```
66360319
```

```
print(vocab[10])
```

```
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w>
```

What the-

```
print(vocab[1010])
```

```
c h a r a c t e r s </w>
```

```
example = len(set(vocab))
print(example)
```

```
191468
```

```
from collections import Counter
```

```
print(type(vocab))
```

```
<class 'list'>
```

```
word_freq = Counter(vocab)
```

```
print(type(word_freq))
```

```
<class 'collections.Counter'>
```

Cant work with this need to make changes

```
vocab_freq = Counter(vocab)
print(len(vocab_freq))
print(type(vocab_freq))
```

```
191468
<class 'collections.Counter'>
```

```
for i, (word, freq) in enumerate(vocab_freq.items()):
    print(word, ":", freq)
    if i == 10:
        break
```

```
S e n j ō </w> : 5
n o </w> : 38479
V a l k y r i a </w> : 53
3 </w> : 8408
: </w> : 3742
U n r e c o r d e d </w> : 1
C h r o n i c l e s </w> : 46
( </w> : 11744
J a p a n e s e </w> : 256
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w> : 3
, </w> : 99786
```

```
"""import re, collections
def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
  for i in range(len(symbols)-1):
    pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out
vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
'n e w e s t </w>':6, 'w i d e s t </w>':3}


num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  print(pairs)
  best = max(pairs, key=pairs.get())
  vocab = merge_vocab(best, vocab)
  print(best)

# print(vocab)"""
```

```
<>:13: SyntaxWarning: invalid escape sequence '\S'
<>:13: SyntaxWarning: invalid escape sequence '\S'
/tmp/ipykernel_14003/4246427281.py:13: SyntaxWarning: invalid escape sequence '\S'
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
'import re, collections\ndef get_stats(vocab):\n  pairs = collections.defaultdict(int)\n  for
word, freq in vocab.items():\n    symbols = word.split()\n  for i in range(len(symbols)-1):\n
pairs[symbols[i],symbols[i+1]] += freq\n  return pairs\n\ndef merge_vocab(pair, v_in):\n  v_o
ut = {}\n  bigram = re.escape(' '.join(pair))\n  p = re.compile(r'(?<!\\S)' + bigram + r'(?!
\\S)')\n  for word in v_in:\n    w_out = p.sub(''.join(pair), word)\n    v_out[w_out] = v_in
[word]\n  return v_out\nvocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,\n'n e w e s t </w>':
6, 'w i d e s t </w>':3}\n\n\nnum_merges = 10\nfor i in range(num_merges):\n  pairs = get_sta
ts(vocab)\n  print(pairs)\n  best = max(pairs, key=pairs.get())\n  vocab = merge_vocab(best,
```

```
for word in range(5):
  subword = vocab[word].split()
  print(vocab[word])
  print(subword)
```

```
S e n j ō </w>
['S', 'e', 'n', 'j', 'ō', '</w>']
n o </w>
['n', 'o', '</w>']
V a l k y r i a </w>
['V', 'a', 'l', 'k', 'y', 'r', 'i', 'a', '</w>']
3 </w>
['3', '</w>']
: </w>
[':', '</w>']
```

```
def get_pair(vocab_freq):

  pairs = {}

  for word,freq in vocab_freq.items():
    subword = word.split()
    for i in range(len(subword)-1):
      pair = (subword[i],subword[i+1])
      pairs[pair] = pairs.get(pair, 0) + freq

  return pairs
```

```
pairs = get_pair(vocab_freq)
```

```
for i, (subword, freq) in enumerate(pairs.items()):
    print(subword, ":", freq)
    if i == 10:
        break
```

```
('S', 'e') : 8242
('e', 'n') : 2054283
('n', 'j') : 30517
('j', 'ō') : 6
('ō', '</w>') : 34
('n', 'o') : 732893
('o', '</w>') : 3358850
('V', 'a') : 1213
('a', 'l') : 1161382
```

```
('l', 'k') : 137616
('k', 'y') : 76296
```

```
print(len(pairs))
```

```
6650
```

```
best = max(pairs, key=pairs.get)
print(best)
```

```
('e', '</w>')
```

```
for i, (word,freq) in enumerate(vocab_freq.items()):
  print(word,type(word), ":", freq)
  if i == 10:
    break
```

```
S e n j ō </w> <class 'str'> : 5
n o </w> <class 'str'> : 38479
V a l k y r i a </w> <class 'str'> : 53
3 </w> <class 'str'> : 8408
: </w> <class 'str'> : 3742
U n r e c o r d e d </w> <class 'str'> : 1
C h r o n i c l e s </w> <class 'str'> : 46
( </w> <class 'str'> : 11744
J a p a n e s e </w> <class 'str'> : 256
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w> <class 'str'> : 3
, </w> <class 'str'> : 99786
```

```
def merge_vocab(word_pair, vocab_in):
      keysub = " ".join(word_pair)
      replacement = "".join(word_pair)

      new_vocab = {}

      for word, freq in vocab_in.items():
            item = 0
            new_token = []
            temp_char_stream = word.split()

            while item < len(temp_char_stream) - 1:
                comparison = temp_char_stream[item] + " " + temp_char_stream[item + 1]

                if comparison == keysub:
                    new_token.append(replacement)
                    item += 2
                else:
                    new_token.append(temp_char_stream[item])
                    item += 1

            final_word = " ".join(new_token)
            new_vocab[final_word] = freq

      return new_vocab
```

```
new_freq = merge_vocab(best, vocab_freq)
```

```
for i,(key,freq) in enumerate(new_freq.items()):
  if "".join(best) in key:
    print(key, ":", freq)
  if i==100:
    break
```

```
J a p a n e s e</w> : 256
t h e</w> : 2980599
o u t s i d e</w> : 49328
r o l e</w> : 608
g a m e</w> : 11638
P o r t a b l e</w> : 20
s a m e</w> : 9939
t i m e</w> : 106451
a r e</w> : 232646
T h e</w> : 636979
l a r g e</w> : 5584
d o n e</w> : 8531
W h i l e</w> : 5512
```

```
print(len(new_freq))
```

```
191468
```

Start coding or generate with AI.